HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

INTERNSHIP

Research on chunking techniques in the RAG problem

DOAN NGOC CUONG
cuong.dn210141@sis.hust.edu.vn

Major: Data Science and AI

Supervisor: PGS.TS Nguyen Thi Kim Anh _____

Chữ kí GVHD

Major: Data Science and AI

School: Information and Communications Technology

HA NOI, 01/2025

ACKNOWLEDGEMENTS

First of all, I would like to express my sincerest and deepest gratitude to Associate Professor, Dr. Nguyen Thi Kim Anh, who has dedicatedly guided and oriented me to overcome challenges during my internship at NLP Lab. My friends and brothers have accompanied me during my internship here.

SUMMARY OF REPORT CONTENT

Nowadays, artificial intelligence and large language models (LLMs) such as ChatGPT, Gemini, and Claude are widely applied in automated question-answering (QA) systems. However, QA systems using LLMs still face significant challenges, including outdated information and hallucinations. To address these issues, the Retrieval-Augmented Generation (RAG) technique was introduced to combine information retrieval, integration of internal data, and updated external information.

Despite its potential, RAG systems still face limitations in terms of input context quality. This study focuses on researching text chunking techniques to enhance the quality of answers in RAG systems. It evaluates chunking methods ranging from traditional to modern approaches, including the application of LLMs, and optimal combination methods.

Student

(Doan Ngoc Cuong)

MỤC LỤC

CHƯƠNG 1. INTRODUCTION 1

1.1 Problem Statement 1

1.2 Current Solutions and Limitations 2

1.3 Contributions of the Research 2

1.4 Structure of the Research 2

CHƯƠNG 2. THEORETICAL BACKGROUND 3

2.1 Problem Context 3

2.2 Large Language Models (LLMs) 3

2.2.1 Overview of Large Language Models 3

2.2.2 Popular LLM Families: Gemini, Qwen, LLaMA 4

2.3 Retrieval-Augmented Generation (RAG) 5

2.3.1 Overview of the Technique 5

2.3.2 Text Chunking 7

2.3.3 User Query-Based Document Retrieval 8

2.3.4 Response Generation Using LLMs 8

2.4 Document Retrieval Strategies 9

2.4.1 Semantic Search: VectorSearch 9

2.4.2 Keyword-Based Search: BM25 10

2.5 Re-ranking Strategies 10

2.6 Related Work 11

2.6.1 Fixed-Size Chunking 11

2.6.2 Semantic Chunking with LlamaIndex's Semantic Chunker 11

2.6.3 RAPTOR 12

CHƯƠNG 3. RESEARCH METHODOLOGY 14

3.1 Overview of Traditional Chunking Methods 14

3.2 Traditional Methods 14

3.2.1 Rule-based Chunking 14

3.2.2 Dynamic Chunking 14

3.2.3 General Evaluation 15

3.3 RAPTOR Methodology 16

3.3.1 Overview of the Methodology 16

3.3.2 Structure and Construction of the RAPTOR Tree 16

3.3.3 Query Strategies in the RAPTOR Tree 17

3.3.4 Advantages and Disadvantages 18

3.3.5 Tree Structure Construction and Optimization 18

3.3.6 Conclusion 19

CHƯƠNG 4. EXPERIMENTAL EVALUATION 21

4.1 Evaluation Parameters 21

4.1.1 BLEU 21

4.1.2 ROUGE-L 21

4.1.3 F1-Score 21

4.1.4 LLM-as-a-Judge 21

4.2 Evaluation Criteria 22

4.3 Experimental Design 22

4.3.1 Evaluation Test Dataset 22

4.3.2 Experimental Configuration 22

4.3.3 Reference Methods 22

4.3.4 Experimental Scenarios 23

4.4 Experimental Results and Observations 23

CHƯƠNG 5. CONCLUSION 26

5.1 Conclusion 26

5.2 Future Directions 26

REFERENCES 29

PHỤ LỤC 31

A. HƯỚNG DẪN VIẾT ĐATN 31

A.1 Quy định chung 31

A.2 Ngành học 32

A.3 Đánh dấu (bullet) và đánh số (numering) 32

A.4 Cách thêm bảng 33

A.5 Chèn hình ảnh 33

A.6 Tài liệu tham khảo 34

A.7 Cách viết phương trình và công thức toán học 34

A.8 Qui cách đóng quyển 34

B. ĐẶC TẢ USE CASE 36

B.1 Đặc tả use case “Thống kê tình hình mượn sách” 36

B.2 Đặc tả use case “Đăng ký làm thẻ mượn” 36

DANH MỤC HÌNH VẼ

Hình 2.1 The Evolutionary Tree of Large Language Models 4
Hình 2.2 Evolution of RAG Methods from 2021 to 2024 6
Hình 2.3 Basic Workflow of a RAG System 7
Hình 2.4 Bi-Encoder Architecture 9
Hình 2.5 Cross-Encoder Architecture for Re-ranking 11
Hình 2.6 RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) 12

Hình 3.1 RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) 16
Hình 3.2 RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) 17

Hình A.1 Internet vạn vật 33
Hình A.2 Qui cách đóng quyển đồ án 35
Hình A.3 Qui cách đóng quyển đồ án 35

DANH MỤC BẢNG BIỂU

Bảng 3.1 Comparison of Chunking Strategies (Updated) 20

Bảng 4.1 Statistics of the **HotPotQA** dataset 22
Bảng 4.2 Kaggle Configuration Information 22
Bảng 4.3 Performance of the proposed solution and reference segmentation techniques on the test dataset 23
Bảng 4.4 Performance of the methods when combined with the RAPTOR technique 24
Bảng 4.5 Performance of segmentation merging methods 24
Bảng 4.6 Performance of retrieval strategies 25

Bảng A.1 Table to test captions and labels. 33

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	**Ý nghĩa**
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHƯƠNG 1. INTRODUCTION

1.1 Problem Statement

Artificial Intelligence (AI) and large language models (LLMs) such as ChatGPT, Gemini, and Claude are becoming increasingly popular, serving as critical tools across various fields, from education and healthcare to business and entertainment. These advancements not only enhance productivity but also significantly improve quality of life through natural language processing and automated content generation. However, despite their potential, LLMs also face notable challenges.

One major issue is the potential for these models to provide outdated or inaccurate information, a phenomenon commonly referred to as "hallucination." This occurs when the model generates responses that sound coherent but contain incorrect information, making it difficult for users to assess reliability /ref. This problem stems from limitations in training data (including internal, outdated, or incomplete data), data inconsistency, and constraints in contextual reasoning.

To address these limitations, the Retrieval-Augmented Generation (RAG) technique was developed as a solution that combines external data retrieval with the response generation capabilities of LLMs /ref. Instead of solely relying on training data, RAG integrates real-time or specialized data from external databases, enhancing the accuracy and timeliness of the responses.

However, the effectiveness of RAG heavily depends on the document chunking process. Suboptimal chunking can result in either loss of context or redundant information, directly impacting retrieval quality and the accuracy of generated answers /ref. Some challenges in chunking include:

- **Loss of Context:** Chunks that are too small may break the connection between key parts of the document.
- **Information Redundancy:** Chunks that are too large force the model to process excessive irrelevant data, reducing accuracy.
- **Defining Chunking Rules:** Selecting the appropriate chunk size and method for specific use cases is complex and non-trivial.

Thus, this research focuses on studying, experimenting with, and evaluating document chunking methods to propose an optimized solution that enhances the effectiveness of RAG systems in information retrieval and response generation.

1.2 Current Solutions and Limitations

The most basic chunking method is fixed-size chunking, which divides documents into equal-sized segments based on a predefined number of sentences or tokens. While this method is simple and computationally efficient, it can easily lead to loss of context or inclusion of unnecessary information, negatively affecting retrieval quality and the accuracy of system responses.

Objectives and Solution Direction This research aims to:

- Study various chunking techniques, evaluate their advantages and disadvantages, and propose an optimized combination.
- Improve the retrieval module's efficiency, particularly in handling long contexts and complex queries.
- Build a prototype system, analyze experimental results, and propose refinements.

The key focuses of this research include:

1. Combining traditional chunking methods with LLMs to improve chunking quality.
2. Designing an efficient retrieval pipeline with additional ranking modules to enhance accuracy.

1.3 Contributions of the Research

This study studies basic chunking methods to improved methods to improve the performance of the RAG system.

1.4 Structure of the Research

The report consists of five chapters:

- Chapter 1: Problem Statement.
- Chapter 2: Theoretical background.
- Chapter 3: Chunking methods are researched in the methodology.
- Chapter 4: Experimental evaluation.
- Chapter 5: Conclusion and Future Directions.

CHƯƠNG 2. THEORETICAL BACKGROUND

2.1 Problem Context

2.2 Large Language Models (LLMs)

2.2.1 Overview of Large Language Models

Large Language Models (LLMs) are advanced AI systems trained on massive amounts of text data to understand and generate natural language. These models typically rely on deep learning architectures, particularly Transformers, to capture complex relationships between words, sentences, and paragraphs. Transformers, first introduced by Vaswani et al. in 2017, have become the foundation of most modern LLMs due to their parallel processing capabilities and effectiveness in understanding long-term context.

LLMs are generally accessible to users and developers through two main approaches:

- **Open-source models:** Models like Qwen and LLaMA fall into the open-source category, allowing users to customize and further develop them based on specific needs. This flexibility supports diverse research and applications. However, deploying these models requires specialized technical knowledge and significant computational resources.
- **Proprietary models:** Models like GPT are proprietary, developed and maintained by large organizations such as OpenAI. These models are user-friendly and often exhibit high quality but are limited in customization capabilities and typically require payment for usage.

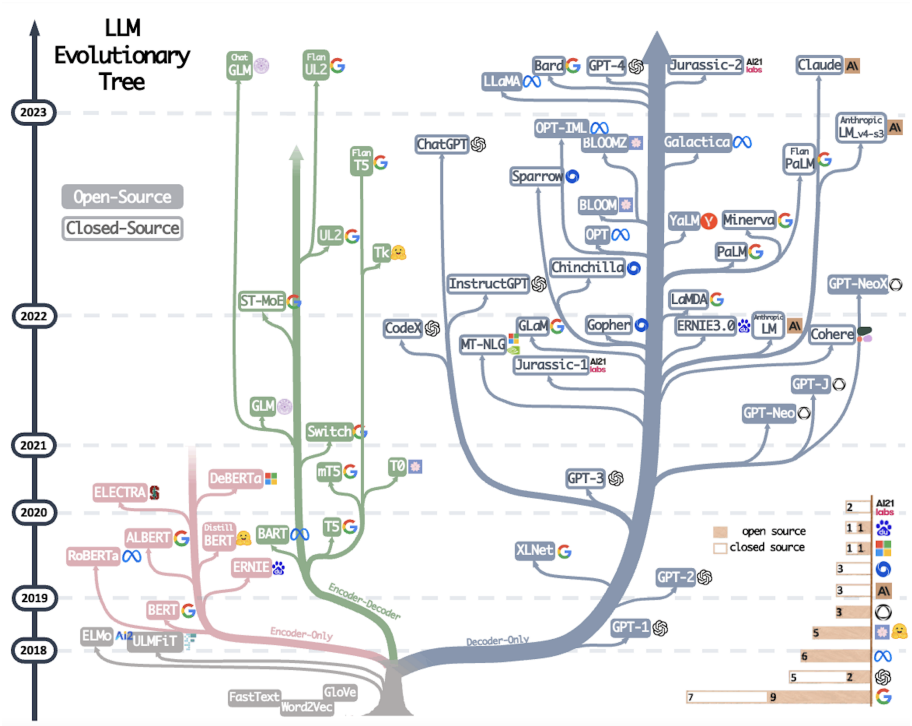
Basic Functionality: LLMs operate by predicting the next word in a sequence of text based on the preceding context. Their training process involves two primary stages: pre-training and fine-tuning. During pre-training, the model learns to predict the next word using a vast and diverse corpus of text, enabling it to grasp linguistic structures and accumulate world knowledge. Fine-tuning adapts the model to specific tasks by further training it on domain-specific datasets.

Applications of Large Language Models: LLMs have demonstrated exceptional capabilities and are widely applied across various domains. One of their most critical applications is in Question-Answering (QA) systems, where they provide accurate and natural responses to user queries. Additionally, they are used for text summarization, generating concise and coherent summaries of lengthy documents, saving time and effort in information processing. In translation, they facilitate

precise and natural language conversion, bridging cultural gaps. Furthermore, LLMs are employed for content creation, generating articles, reports, or other creative content, opening new opportunities in media and content industries.

Challenges: Despite their significant benefits and increasing versatility, LLMs have certain limitations, such as the inability to update knowledge after the training process is complete. As models only know information up to their training date, integrating new knowledge requires retraining or fine-tuning, which is time-consuming and resource-intensive. Ensuring the quality and accuracy of new data is also a substantial challenge.

The Evolution of Large Language Models:



Hình 2.1: The Evolutionary Tree of Large Language Models

Figure 2.1 illustrates the rapid and explosive growth of LLMs. From 2023 onwards, the number of LLMs has surged significantly, reflecting global interest and predicting this trend as the future of AI technology.

2.2.2 Popular LLM Families: Gemini, Qwen, LLaMA

Gemini: Developed by Google DeepMind, Gemini focuses on commercial applications with continuous learning capabilities and optimized natural language processing. It integrates deeply into Google's services, offering intelligent solutions for businesses and individuals. Notable versions include Gemini 1.5 and Gemini Pro, each improving performance and features over its predecessors.

Qwen: Developed by Alibaba, Qwen caters to businesses with multilingual support and efficient information management. It is designed to meet natural language processing needs in business environments, providing robust tools for data analysis and customer interaction. Versions like Qwen-Enterprise and Qwen-MultiLingual have been widely deployed in e-commerce and customer service applications.

LLaMA: An open-source model from Meta, LLaMA supports research and development in AI without dependence on proprietary platforms. New versions like LLaMA3.2 significantly enhance performance, expanding applications across various domains, from academic research to industrial use. LLaMA stands out for its flexibility and high customizability, enabling developers to integrate it seamlessly into existing systems.

These models exemplify ongoing advancements in AI, with each model family offering unique features and applications to meet diverse user and business needs. The continuous development of LLMs not only improves their understanding and generation of natural language but also opens up numerous opportunities for applying AI across industries.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an advanced AI technique that combines the ability to retrieve information from external data sources with the natural language generation capabilities of LLMs. This method aims to enhance the accuracy and reliability of QA systems by providing supplementary and updated information from external documents, helping LLMs avoid hallucination and delivering answers based on real-world data.

2.3.1 Overview of the Technique

RAG was developed to address some limitations of LLMs in handling questions that require detailed, updated, or domain-specific information. One major challenge for LLMs is hallucination, where models generate coherent but incorrect answers based on false or non-existent information. This issue arises because LLMs rely solely on knowledge learned during training, which is limited by the timing and content of the training data.

RAG resolves this issue by incorporating external information retrieval capabilities, allowing models to access updated information and enhance the reliability of their answers. Instead of relying solely on intrinsic knowledge, RAG leverages both retrieval capabilities from external databases or documents and language generation. This improves accuracy and extends the applicability of LLMs in tasks such as deep QA, data analysis, and decision support.

The Evolution of RAG Methods: The Evolution of RAG Methods:

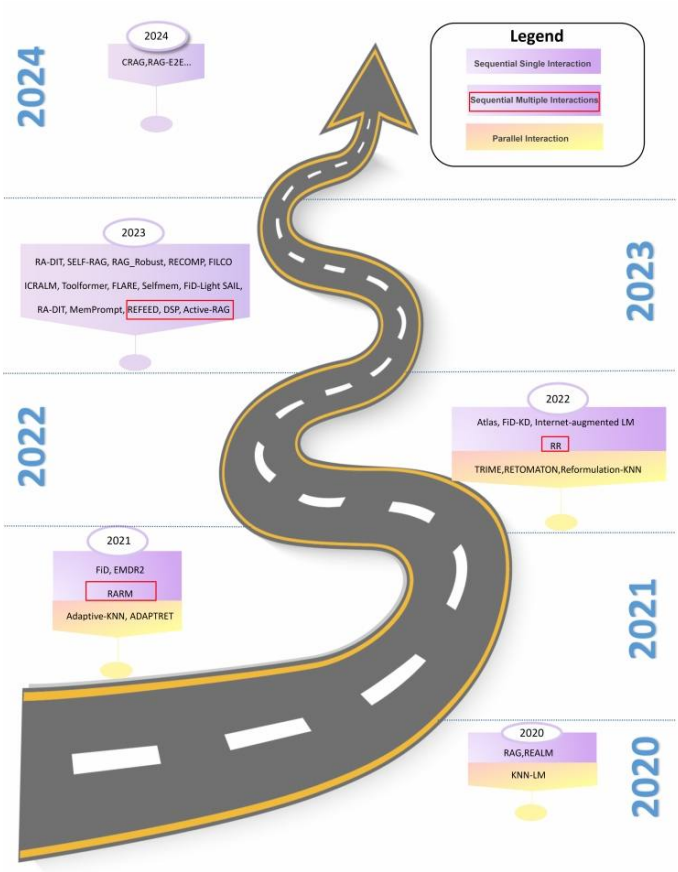


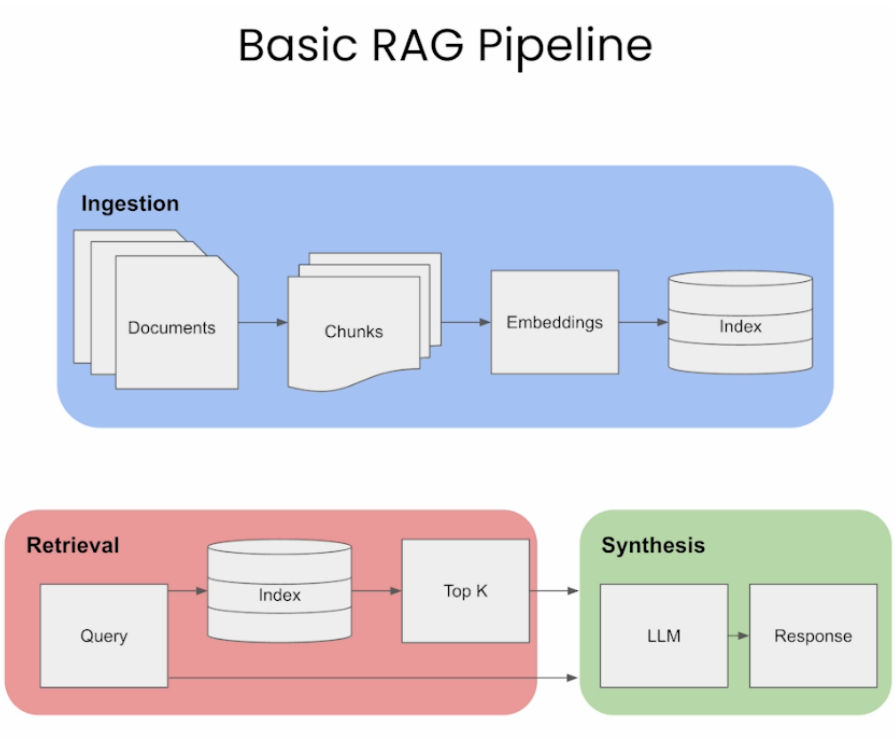
Figure 3: A roadmap of the three types of interactions. The purple areas represent work on Sequential Interaction RALM models, the red boxes signify work on Sequential Multiple Interactions RALMs models, and the yellow areas indicate work on Parallel Interaction RALM models.

Hình 2.2: Evolution of RAG Methods from 2021 to 2024

The evolution of Retrieval-Augmented Generation (RAG) methods, as depicted in Figure 2.2, illustrates significant advancements from 2020 to 2024. Starting with simpler methods such as RAG-REALM in 2020, the roadmap shows a clear trend toward more complex and specialized approaches like CRAG and RAG-E2E in 2024. This progression highlights three distinct interaction types: Sequential Single Interaction (purple), Sequential Multiple Interactions (red), and Parallel Interaction (yellow). Early models focused on straightforward processes, while later iterations incorporated iterative refinements and simultaneous processing to enhance efficiency and contextual understanding. By 2023, methods like Active-RAG and REFEED emerged, emphasizing dynamic interaction and domain-specific optimization. The diversification and innovation in RAG approaches reflect a growing need for systems that excel in accuracy, adaptability, and the integration of external knowledge, paving the way for more robust and efficient retrieval and generation frameworks.

Basic Architecture of RAG Systems:

The RAG system consists of three main modules operating in a sequential flow, as illustrated in Figure 2.3:



Hình 2.3: Basic Workflow of a RAG System

In the RAG architecture, each module plays a crucial role in ensuring the system delivers accurate and reliable answers. The text chunking module optimizes retrieval efficiency while reducing the load for subsequent processing steps. The document retrieval module ensures that the system only processes the most relevant text segments to the query, enhancing both speed and accuracy. Finally, the response generation module integrates information from retrieved segments with user queries to generate answers, ensuring results are both accurate and natural.

This structure allows RAG to overcome LLM limitations and lays the groundwork for various intelligent applications, such as virtual assistants, advanced QA systems, and research support.

2.3.2 Text Chunking

Text chunking is a crucial step in information processing that involves splitting original documents into smaller, logically coherent segments. The primary goal is to increase retrieval efficiency by reducing the size of data to be processed, ensuring that the system focuses only on information most relevant to the query. This not only reduces computational load but also preserves semantic and contextual relationships essential for accurate understanding.

Common chunking methods include fixed-size chunking, semantic chunking, and LLM-based chunking. Fixed-size chunking is a straightforward method that divides documents into equal-sized segments based on sentence or token count. However, this method may disrupt semantic relationships, leading to the loss of critical information. Semantic chunking, on the other hand, focuses on maintaining semantic coherence between sentences but requires complex analysis and higher computational resources. Modern methods use LLMs to determine optimal chunking points based on context and linguistic structure, offering flexibility and high accuracy.

Chunking typically begins by splitting the document into individual sentences. These sentences are then grouped into segments based on criteria such as maximum length or semantic context. The final text chunks are indexed and stored in a database, ready for retrieval. Despite advantages like improved processing efficiency and context retention, chunking methods still face challenges in balancing semantic integrity and computational efficiency.

2.3.3 User Query-Based Document Retrieval

In a RAG system, user query-based document retrieval is a critical process for identifying and selecting relevant information. This process involves two main steps: retrieval and re-ranking. The goal is to ensure that only the most pertinent text segments related to the user's query are selected for input into the LLM.

The first step is retrieval, where algorithms identify relevant text segments in the database. Two common methods are Vector Search and BM25. Vector Search converts text into numerical vectors and compares their similarity to find relevant segments. This method is particularly effective for understanding contextual and semantic relationships between words and sentences. In contrast, BM25 is a traditional search algorithm based on term frequency and inverse document frequency, often used for basic query processing.

Following retrieval, the results are re-ranked based on their relevance and quality. Re-ranking uses machine learning models or advanced algorithms to prioritize the most significant text segments. This step improves accuracy and ensures the system delivers appropriate and reliable responses.

2.3.4 Response Generation Using LLMs

After retrieving relevant text segments, the next step is generating responses using LLMs. This stage involves combining information from retrieved text with user queries to produce accurate and natural answers. The quality of prompts designed to guide the LLM in generating responses significantly impacts the process.

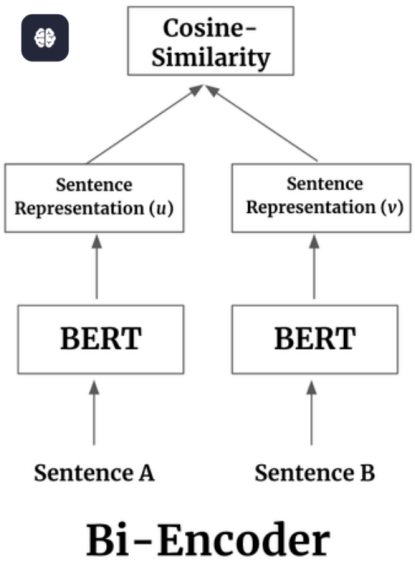
Prompts are carefully crafted text inputs that integrate information from documents and user queries. They provide context and direction for the LLM, ensuring it processes queries accurately and delivers appropriate responses. Effective prompts are clear, precise, and sufficiently detailed to help the model understand the task and generate relevant answers. Crafting prompts (Prompt Engineering) requires skill and deep understanding of how LLMs process natural language to optimize response quality.

During response generation, the system first creates a prompt by combining the user's query with retrieved text segments. This prompt is then sent to the LLM to generate an answer. Finally, the result is reviewed and processed if necessary before being returned to the user. This process ensures that answers are accurate while maintaining natural language flow and coherence.

However, generating responses with LLMs also presents challenges. Poorly designed prompts can lead to inaccurate or irrelevant answers. Additionally, ensuring that the model accurately interprets and processes the context of the query is particularly challenging in cases requiring complex or multi-layered information.

2.4 Document Retrieval Strategies

2.4.1 Semantic Search: VectorSearch



Hình 2.4: Bi-Encoder Architecture

Figure 2.4 illustrates how Bi-Encoder is used in VectorSearch to generate semantic vector representations for queries and documents. This process measures the similarity between vectors, enabling efficient information retrieval based on meaning rather than keywords.

Method Overview: VectorSearch is a search technique that measures semantic similarity between queries and documents in a database. It uses embedding models such as BERT or Sentence Transformers to represent both queries and documents as vectors in a multi-dimensional space. Similarity between these vectors, often measured using cosine similarity, determines relevance.

Advantages and Disadvantages: VectorSearch excels at retrieving information based on contextual meaning, overcoming limitations of traditional keyword-based methods. However, it requires significant computational resources and heavily depends on embedding quality, which can be challenging for large or heterogeneous datasets. Moreover, vector space distance may not always accurately reflect semantics in complex scenarios.

2.4.2 Keyword-Based Search: BM25

Method Overview: BM25 (Best Matching 25) is a search algorithm based on term frequency (TF) and inverse document frequency (IDF). This method evaluates document relevance by assessing the occurrence of query keywords in the document, with weighting adjustments to optimize search performance.

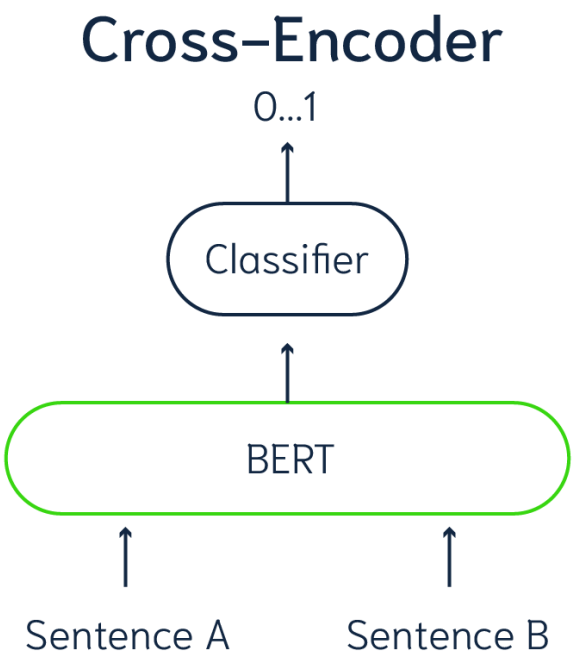
Advantages and Disadvantages: BM25 is easy to implement, computationally efficient, and suitable for systems with large datasets. It performs well for straightforward queries where keywords are clearly present in the document. However, BM25 cannot understand deep semantic context and may miss relevant documents if keywords are not explicitly mentioned. This makes it less effective for complex languages or multi-layered queries.

2.5 Re-ranking Strategies

Figure 2.6 illustrates the Cross-Encoder mechanism used in the re-ranking module to evaluate the relevance between queries and documents. The model processes query-document pairs simultaneously, leveraging semantic and syntactic information to accurately reorder search results.

Method Overview: Document re-ranking is an advanced step performed after initial retrieval of documents. This process re-evaluates document relevance using advanced criteria or deep learning models to optimize the order of search results. Re-ranking models are designed to deeply understand semantics and syntax, providing results more aligned with user queries.

Advantages and Disadvantages: Re-ranking significantly improves search quality, particularly for complex queries or high-precision requirements. This method leverages deep learning models to process semantic and syntactic factors, yielding highly



Hình 2.5: Cross-Encoder Architecture for Re-ranking

relevant results. However, re-ranking requires substantial computational resources and may increase query processing time. Additionally, optimizing the model for specific scenarios demands time and specialized knowledge, complicating deployment at scale.

2.6 Related Work

This section reviews prominent research related to text chunking and various approaches in natural language processing. Each method is summarized and evaluated for its strengths and weaknesses to assess applicability to the current problem.

2.6.1 Fixed-Size Chunking

Method Overview: This approach divides a text into chunks of fixed size, usually determined by the number of words or characters. Each chunk is of equal size, regardless of content or meaning.

Evaluation: This method is simple, easy to implement, and has fast processing speeds, making it suitable for large datasets. However, it often cuts across sentences or meanings, leading to fragmented content and is unsuitable for texts with complex semantic structures.

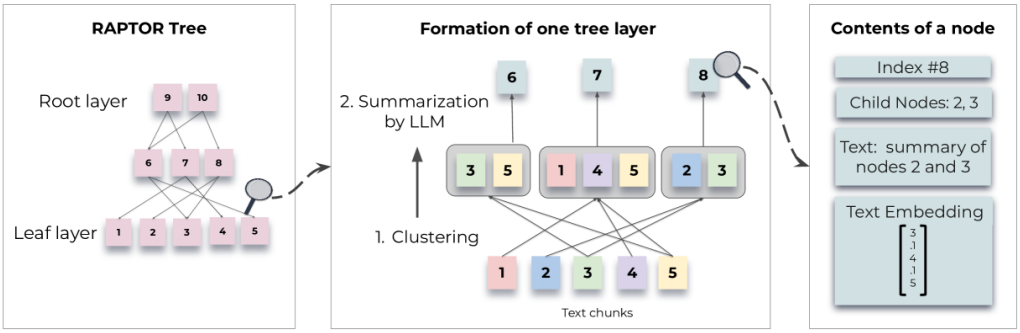
2.6.2 Semantic Chunking with LlamaIndex's Semantic Chunker

Method Overview: The Semantic Chunker in LlamaIndex is an advanced tool for segmenting text based on semantic meaning. Instead of dividing text into fixed chunks, this tool uses semantic embedding models to identify split points between

sentences, ensuring each chunk contains semantically related sentences. This preserves the text's coherence and core meaning, making it particularly useful in natural language processing applications.

Evaluation: Semantic Chunker preserves semantic integrity and improves performance in tasks like information retrieval and QA. However, it has drawbacks, such as inflexibility in handling heterogeneous texts and generating unevenly sized chunks, making control and further processing challenging. Additionally, it requires significant computational resources and depends heavily on the quality of semantic embedding models, increasing implementation complexity.

2.6.3 RAPTOR



Hình 2.6: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

Method Overview: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) is a modern approach combining information retrieval and text generation. RAPTOR's key feature is its hierarchical tree structure built recursively from the bottom up. Specifically, initial text segments are split, embedded into vector space, grouped into clusters based on semantics, and summarized into parent nodes. This process repeats iteratively, creating a hierarchical tree of information with varying levels of abstraction, from detailed to general.

Evaluation: RAPTOR demonstrates outstanding effectiveness in handling complex information processing and multi-step reasoning tasks. Its hierarchical tree structure enables the integration of information from multiple contextual levels, allowing the model to deeply understand content and provide accurate, comprehensive answers. This is especially useful in tasks like QA on long texts or deep semantic analysis. However, RAPTOR has some limitations. Building and summarizing recursively requires substantial computational resources, leading to higher costs than traditional retrieval methods. Moreover, RAPTOR's complex structure demands expertise in natural language processing and systems implementation.

Chapter Summary

This chapter presented an overview of critical components in QA systems based on Retrieval-Augmented Generation (RAG). Firstly, LLMs were introduced as the foundational technology with their natural language processing and generation capabilities, while also highlighting limitations like hallucination and knowledge constraints at training time. Next, the basic workflow of RAG was analyzed, including its three main modules: text chunking, document retrieval, and response generation.

Additionally, text chunking methods were thoroughly examined, from fixed-size chunking to semantic chunking and LLM-based chunking. While each method has its strengths, such as simplicity and efficiency (fixed-size chunking) or maintaining semantic coherence (semantic chunking), they also exhibit limitations. For example, fixed-size chunking may lose critical context, whereas semantic chunking demands computational complexity and significant resources.

Document retrieval strategies were also discussed, including semantic retrieval via VectorSearch, keyword-based retrieval with BM25, and re-ranking strategies to enhance retrieval quality. Each method's advantages and disadvantages were clearly outlined in this chapter.

Overall, challenges like context loss during chunking, semantic retrieval complexity, and high computational costs will be addressed in subsequent chapters through proposed improvements. These refinements aim to optimize system efficiency, increase accuracy, and enhance the practical applicability of RAG-based QA systems.

CHƯƠNG 3. RESEARCH METHODOLOGY

3.1 Overview of Traditional Chunking Methods

Table 3.1 provides a comparative overview of various traditional chunking strategies, highlighting their descriptions, advantages, and limitations. This table serves as a comprehensive reference to understand the trade-offs between simplicity, contextual preservation, and computational efficiency associated with each method.

3.2 Traditional Methods

In this section, I will present the baseline methods that serve as the foundation for subsequent chunking approaches. These methods fall into two main categories: *rule-based chunking* and *dynamic chunking*, with the latter incorporating semantic similarity models and large language models (LLMs).

3.2.1 Rule-based Chunking

Rule-based chunking divides long texts into chunks of fixed length without considering sentence boundaries. However, the **Llama Index** method (Langchain, 2023) improves upon this by balancing the preservation of sentence boundaries while ensuring that the token count in each segment remains close to a preset threshold.

Key Features:

- Splits text based on a fixed number of tokens.
- Ensures sentence integrity within chunks.

3.2.2 Dynamic Chunking

Dynamic chunking relies on deep learning models designed to adapt to the context and structure of the text.

a, Similarity Chunking

This method uses sentence embedding models to calculate the semantic similarity between sentences, then selects segmentation points based on a similarity threshold. Sentences within each chunk exhibit high semantic coherence (Xiao et al., 2023).

Advantages:

- Segments text based on contextual meaning instead of length.
- Effectively groups sentences with high semantic relatedness.

b, Dense X Retrieval

This method introduces a new segmentation granularity called *proposition*, where the text is divided into the smallest units conveying complete information (Chen et al., 2023a). It is particularly effective for fact-based content like Wikipedia but faces limitations with sequential or narrative content.

c, LumberChunker

LumberChunker (Duarte et al., 2024) leverages the reasoning capabilities of large language models (LLMs) to predict optimal segmentation points within the text. However, this method demands high instruction-following capabilities from LLMs, resulting in significant resource costs when using models like Gemini.

d, Margin Sampling Chunking (MSP)

This method was proposed to optimize LumberChunker, improving its adaptability to smaller models. The text is split into a collection of sentences $(x_1, x_2, ..., x_n)$, and segmentation points are determined based on the probability difference (*Margin*):

$$\text{Margin}_M(x_i) = P_M(y = k_1|\text{Prompt}(x_i, X')) - P_M(y = k_2|\text{Prompt}(x_i, X'))$$

Where:

- k_1, k_2: Decision to segment or not.
- Prompt(x_i, X'): Instruction guiding the model to decide whether x_i and X' should be merged.

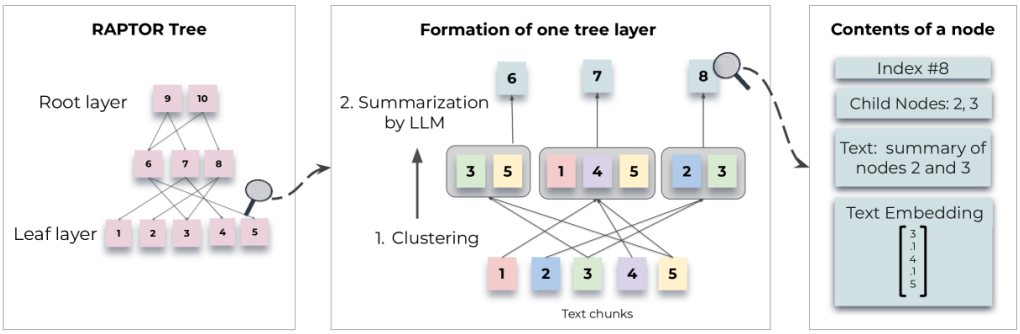
The Margin$_M(x_i)$ is compared to a threshold θ to decide whether to segment. The threshold is initially set to 0 and adjusted based on the average historical Margin$_M(x_i)$.

3.2.3 General Evaluation

Baseline methods provide a foundation for text segmentation, each with unique strengths:

- Rule-based chunking is simple and fast but lacks the ability to capture deep semantic understanding.
- Dynamic chunking offers higher efficiency in organizing information but demands greater computational resources.

These aspects will be compared in detail with Meta-Chunking in subsequent sections.



Hình 3.1: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

3.3 RAPTOR Methodology

3.3.1 Overview of the Methodology

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) is an advanced method in the field of retrieval-augmented generation (RAG). This approach combines information retrieval and text generation by constructing a recursive hierarchical tree structure from the bottom up.

The RAPTOR tree-building process starts by segmenting input documents into shorter text passages, each containing approximately 100 words to ensure consistency and maintain reasonable context. These text passages are then transformed into semantic embedding vectors using the SBERT (Sentence-BERT) model, allowing the representation of semantic relationships between text passages.

Next, these embedding vectors are clustered based on semantic similarity using the Gaussian Mixture Models (GMM) algorithm, ensuring that text passages in the same cluster share similar meanings. Each cluster is then summarized into a parent node using the GPT-3.5-turbo language model, forming different abstraction layers in the tree. This process is recursively repeated until clustering is no longer possible, creating a hierarchical tree structure with various abstraction levels ranging from detailed to general (Source: arXiv 2401.18059v1).

3.3.2 Structure and Construction of the RAPTOR Tree

a, Segmentation and Vector Embedding

The RAPTOR tree construction process begins by segmenting documents into smaller text passages, each containing around 100 words. This segmentation not only helps maintain contextual consistency but also optimizes the embedding and clustering processes. These text passages are embedded into vector space using SBERT (Reimers & Gurevych, 2019), enabling the model to understand and represent semantic relationships between text passages.

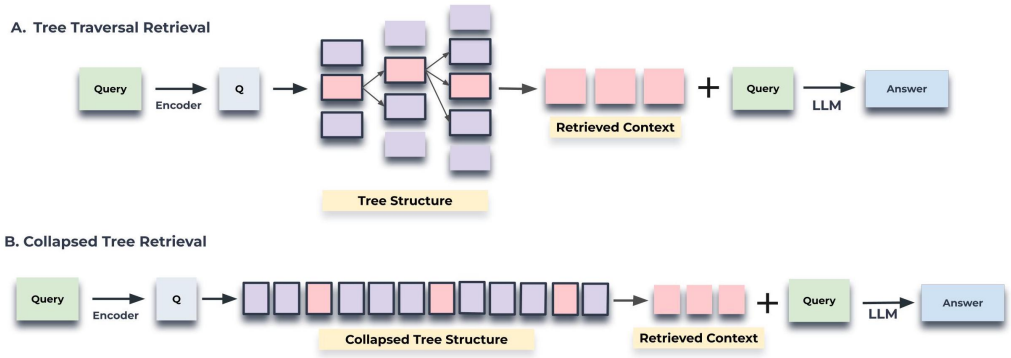
b, Semantic Clustering

Once the embedding vectors are generated, RAPTOR employs Gaussian Mixture Models (GMM) to cluster the text passages based on semantic similarity. GMM allows each text passage to belong to multiple clusters, enabling greater flexibility in grouping text passages related to multiple topics. To enhance clustering efficiency in high-dimensional vector space, RAPTOR applies dimensionality reduction using Uniform Manifold Approximation and Projection (UMAP), preserving semantic structure while reducing computational costs.

c, Summarization and Tree Structure Construction

After clustering, each cluster is summarized into a parent node using GPT-3.5-turbo. This summarization process reduces the size of the data to be processed while retaining core information. These parent nodes then serve as child nodes for the next clustering step, and the process is repeated until further clustering is not possible. The result is a hierarchical tree structure where each layer represents a different level of abstraction of the information, from detailed to general.

3.3.3 Query Strategies in the RAPTOR Tree



Hình 3.2: RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

RAPTOR supports two main query strategies: **Tree Traversal** and **Collapsed Tree Retrieval**.

a, Tree Traversal

This approach starts from the root layer of the tree and traverses child nodes based on similarity to the query. At each layer, RAPTOR selects the top-k nodes with the highest similarity scores and continues accessing their child nodes. This process repeats until the leaf layer is reached. Finally, all information from selected nodes across different layers is aggregated to form the query context for the large language model (LLM).

b, Collapsed Tree Retrieval

This approach collapses all layers of the RAPTOR tree into a single layer and performs similarity search directly across all nodes. RAPTOR then selects the nodes with the highest similarity scores until a predefined token limit is reached. This method is more flexible as it allows nodes at any abstraction level to be selected based on the query, but it requires efficient search libraries like FAISS to reduce computation time.

3.3.4 Advantages and Disadvantages

a, Advantages

RAPTOR offers several advantages that make it a valuable tool in advanced information retrieval systems. First, its hierarchical tree structure enables access to information at varying levels of detail, enhancing the system's ability to understand and respond to complex questions effectively.

Additionally, RAPTOR demonstrates exceptional performance in tasks requiring multi-step reasoning, significantly outperforming traditional methods on datasets such as QuALITY and NarrativeQA. The scalability of RAPTOR is another strength, allowing it to efficiently handle large datasets, which makes it suitable for applications such as document summarization and in-depth question answering. Furthermore, by incorporating external information into its retrieval process, RAPTOR minimizes hallucinations in large language models, thus ensuring greater reliability and precision in responses.

b, Disadvantages

Despite its advantages, RAPTOR also presents several challenges. The method incurs high computational costs, as it requires substantial resources for operations like vector embedding, clustering, and summarization. Additionally, implementing RAPTOR demands significant expertise in natural language processing, machine learning, and data organization, necessitating a skilled and experienced team. Another limitation is its latency in real-time applications; the processing times involved in building and maintaining the hierarchical tree structure make RAPTOR unsuitable for scenarios requiring instant responses.

3.3.5 Tree Structure Construction and Optimization

a, Embedding and Clustering

The RAPTOR tree-building process begins by transforming text passages into semantic embedding vectors using SBERT. These vectors are then dimensionally reduced with UMAP to enhance clustering efficiency. Gaussian Mixture Models

(GMM) are used to cluster text passages based on semantic similarity, allowing each passage to belong to multiple clusters.

b, Summarization and Parent Node Construction

After clustering, each cluster is summarized into a parent node using GPT-3.5-turbo. These parent nodes represent the main topics or ideas of the text clusters and become child nodes for the next clustering step. This process repeats until further clustering is not possible, forming a hierarchical tree with multiple abstraction levels.

c, Efficient Query Strategies

RAPTOR supports two main query strategies: Tree Traversal and Collapsed Tree Retrieval. Tree Traversal traverses from root to leaf, selecting top-k nodes at each layer based on similarity to the query. Collapsed Tree Retrieval collapses the entire tree into a single layer and selects top-k nodes with the highest similarity scores. This method has been shown to be more effective on datasets like QASPER due to its flexibility in selecting nodes at appropriate abstraction levels.

3.3.6 Conclusion

RAPTOR represents a significant advancement in information retrieval and text generation systems, particularly effective in tasks requiring long-context or complex reasoning. With its hierarchical tree structure, this method not only enhances semantic understanding but also broadens the applicability of information retrieval systems. However, the computational resource demands and implementation complexity remain major challenges, requiring careful consideration for real-world deployment.

Bảng 3.1: Comparison of Chunking Strategies (Updated)

Chunking Strategy	**Description**	**Advantages**	**Disadvantages**
A. Fixed-size Chunking	Splits the text into chunks of fixed length with overlap.	Easy to implement, uniform chunk sizes.	May cut across sentences and paragraphs, potentially losing important context.
B. Sentence-based Chunking	Splits text based on complete sentences, preserving context.	Maintains context, suitable for content with complex structures.	Chunks may vary in length, which could lead to overly long chunks.
C. Hybrid Chunking	Combines Fixed-size and Sentence-based methods, prioritizing splits at sentence boundaries.	Better context preservation compared to Fixed-size, especially useful for sentence-based content.	More complex to implement.
D. Semantic-based Chunking	Uses language models to segment text based on semantic meaning instead of fixed length.	Optimized for semantic search, better preservation of meaning.	Requires significant computational resources, challenging to implement.
E. Recursive Character-based Splitting	Recursively splits text using delimiters such as punctuation marks.	Maintains structure better than simple character splitting, suitable for well-structured text.	May cut across meaning if delimiters are not well-aligned.
F. Agentic Chunker	Utilizes "agents" to automatically group related propositions into semantic chunks.	Automatically optimizes based on context, more flexible and intelligent.	Complex, requiring advanced AI models.
I. Document-type-based Splitting	Splits based on document types (PDF, Markdown, Python code).	Optimized for specific document types, better semantic preservation.	Requires document classification before splitting.

CHƯƠNG 4. EXPERIMENTAL EVALUATION

In this chapter, I will present experimental results to compare and provide an overview of the methods I have discussed and proposed. In the first part, I will present the setup of evaluation parameters (Section 4.1) and evaluation criteria for experiments (Section 4.2). In the next part, I will discuss the selection of baselines for comparison (Section 4.3.3) as well as provide experimental approaches (Section 4.3.4). The remaining sections will detail each experiment.

4.1 Evaluation Parameters

Question-answering systems using large language models are typically evaluated in two ways: automatic metrics and model-based evaluations. Below, I present some automatic metrics and model-based evaluation methods that I have used to assess the quality of the system's output:

4.1.1 BLEU

BLEU (Bilingual Evaluation Understudy) **papineni-etal-2002-bleu** is one of the most popular automatic metrics used to evaluate the quality of question-answering systems. BLEU operates by comparing n-grams (sequences of n consecutive words) in the system's output sentences with the n-grams in the reference sentences. BLEU typically uses from 1-gram to 4-gram to ensure evaluation of both grammar and meaning.

4.1.2 ROUGE-L

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence) **lin-2004-rouge** is a popular automatic evaluation metric that focuses on finding the longest common subsequence between the system's answer and the reference answer. ROUGE-L helps assess the degree of matching in structure and word order between the two answers.

4.1.3 F1-Score

F1-Score is the harmonic mean of Precision and Recall, providing a balanced view of the system's ability to answer questions accurately and completely.

4.1.4 LLM-as-a-Judge

Large Language Models as a Judge **zheng2023judgingllmasajudgemtbenchchatbot** is an evaluation method that uses large language models to analyze and compare the system's answers with reference answers, based on their ability to understand context and meaning.

4.2 Evaluation Criteria

The evaluation criteria include:

1. Answer quality, measured by BLEU, ROUGE-L, F1, and LLM-Score.
2. Answer generation time and resource usage.
3. Text segmentation time.

4.3 Experimental Design

4.3.1 Evaluation Test Dataset

The system is evaluated on the **HotPotQA** dataset, consisting of 200 text passages from Wikipedia and 200 pairs of questions and answers.

Statistics	Avg. Len
Average Text Length	9151
Average Question Length	19.525
Average Answer Length	3.775

Bảng 4.1: Statistics of the **HotPotQA** dataset

Table 4.1 shows that the test dataset has significantly different text and question lengths, providing a basis for evaluating segmentation and retrieval techniques.

4.3.2 Experimental Configuration

The experimental configuration uses the Kaggle platform with a Tesla P100 GPU, Xeon CPU, 29GB RAM, and 73GB of available disk space.

System	Information
GPU	1 x Tesla P100, 3584 CUDA cores, 16GB GDDR6 VRAM
CPU	1 x single core hyper-threaded Xeon Processor @2.2Ghz (1 core 2 threads)
RAM	∼29 GB Available
Disk	∼73.1 GB Available

Bảng 4.2: Kaggle Configuration Information

Table 4.2 provides a detailed description of the experimental configuration, ensuring feasibility and performance for the scenarios.

4.3.3 Reference Methods

The proposed segmentation solution is compared with other segmentation techniques such as:

1. **Fixed-size Chunking (FSC):** Segmenting text based on a fixed number of tokens, combined with *overlap* to maintain context between segments.
2. **Breakpoint-based Semantic Chunking (BSC):** Semantic segmentation based

on stop points, using the LlamaIndex library.

3. **LLM-based Chunking (LBC):** Segmentation using large language models to determine semantic breakpoints.
4. **RAPTOR:** Combines segmentation, semantic clustering, and cluster summarization using the Gemini-1.5-flash model.

4.3.4 Experimental Scenarios

The experimental scenarios include:

1. Comparing the performance of different segmentation techniques.
2. Evaluating the effectiveness of RAPTOR combined with segmentation techniques.
3. Testing segmentation merging methods.
4. Comparing different retrieval and reranking strategies.

4.4 Experimental Results and Observations

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	ROUGE-L	F1	LLM-Score
PPDX	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**
RAPTOR	42.82	24.02	16.79	12.32	51.35	50.5	60
LBC	41.82	24.69	16.56	12.42	49.48	49.11	56.5
BSC	43.89	25.9	18.83	13.57	51.99	51.53	62
FSC	42.48	25	17.22	12.24	50.10	49.31	57

Bảng 4.3: Performance of the proposed solution and reference segmentation techniques on the test dataset

Table 4.3 shows that the proposed solution (PPDX) achieves the highest performance across all metrics. Compared to the BSC technique, PPDX increases ROUGE-L by 6.5% and F1 by 4.88%, while reducing the average number of input tokens, demonstrating the effectiveness of segmentation merging and the use of large language models. The RAPTOR technique also improves the performance of other methods, especially when combined with BSC.

Table 4.4 shows that combining RAPTOR with segmentation techniques such as FSC, BSC, and LBC significantly improves performance, especially when applied to BSC.

Table 4.5 demonstrates that merging 400 tokens achieves the best performance, balancing the metrics and input context.

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	ROUGE-L	F1	LLM-Score
FSC	42.48	25	17.22	12.24	50.10	49.31	57
RAPTOR (FSC)	**42.82**	24.02	**16.79**	**12.32**	**51.35**	**50.5**	**60**
BSC	43.89	25.9	18.83	13.57	51.99	51.53	62
RAPTOR (BSC)	**44.01**	**26.33**	18.26	13.07	**53.89**	**53.12**	**62.5**
LBC	41.82	24.69	16.56	12.42	49.48	49.11	56.5
RAPTOR (LBC)	**42.60**	**24.90**	**16.57**	**12.80**	**50.41**	**49.52**	**59**

Bảng 4.4: Performance of the methods when combined with the RAPTOR technique

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	ROUGE-L	F1	LLM-Score
No Merge (RAPTOR(LBC))	42.6	24.9	16.57	12.8	50.41	49.52	59
Merge 200 tokens	42.88	25.51	17.72	12.85	50.79	50	59
Merge 300 tokens	45.07	26.77	19.03	13.67	53.68	52.99	62.5
Merge 400 tokens (PPDX)	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**
Merge 500 tokens	45.81	27.89	19.52	14.3	53.59	52.94	62

Bảng 4.5: Performance of segmentation merging methods

Table 4.6 describes the results when comparing the performance of different retrieval and reranking strategies. The Hybrid Search + Rerank (PPDX) method significantly improved performance compared to the other methods, especially in ROUGE-L and F1.

Conclusion of the Chapter

This chapter has provided a detailed presentation of the experimental design and offered evaluation and comparison information between the proposed solution and the reference methods. The data from the metrics indicate that the proposed solution has produced higher-quality answers compared to other reference techniques. However, the use of multiple large language models and techniques that are only

	Bleu-1	Bleu-2	Bleu-3	Bleu-4	ROUGE-L	F1	LLM-Score
Vector Search + Rerank	41.57	24.24	16.43	12.23	49.68	48.1	55.5
BM25 + Rerank	42.04	26.11	18.15	12.97	51.30	49.44	59.5
HybridSearch + Rerank (PPDX)	**46.64**	**28.32**	**19.93**	**14.41**	**55.41**	**54.07**	**63.5**

Bảng 4.6: Performance of retrieval strategies

evaluated based on the quality of the answers generated by large language models have not provided the most accurate assessment of the impact of different document segmentation techniques on the answers. This issue, along with many other unresolved issues in the report, will be clearly stated and future development directions will be outlined in Chapter 5 - the final chapter of the research internship report.

CHƯƠNG 5. CONCLUSION

5.1 Conclusion

This thesis addressed key challenges in text chunking for RAG systems, evaluating various methods to improve retrieval and generation accuracy. However, some limitations, such as computational costs and optimization for diverse datasets, remain.

5.2 Future Directions

Future work could focus on optimizing chunking techniques for specific domains, improving computational efficiency, and exploring advanced integration of LLMs with retrieval systems for enhanced performance.

NOTES ON REFERENCES

arxiv2024 RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval, *arXiv*, January 31, 2024. Available: https://arxiv.org/html/2401.18059v1.

educative2024 Educative, “Mastering RAG with RAPTOR: A comprehensive guide using LlamaIndex," Available: https://www.educative.io/blog/mastering-rag-with-raptor, accessed September 6, 2024.

medium2024 R. Patekar, “Mastering the 25 Types of RAG Architectures: When and How to Use Each One," Medium, November, 2024. Available: https://medium.com/@rupeshit/mastering-the-25-types-of-rag-architectures-when-and-how-to-use-each-one-2ca0e4b944d7.

langchain2024 LLM Langchain Blogs, “Efficient Information Retrieval RAG for Complex PDFs Using RAPTOR," July 10, 2024. Available: https://www.langchain.ca/blog/efficient-information-retrieval-from-complex-pdfs-using-raptor-rag/.

csdn2024 CSDN Blog, “RAPTOR: RAG," July 4, 2024. Available: https://blog.csdn.net/qq_41739364/article/details/138336034.

deeplearning2024 DeepLearning.AI, “Richer Context for RAG: RAPTOR, a recursive summarizer, captures more relevant context," May 29, 2024. Available: https://www.deeplearning.ai/the-batch/raptor-a-recursive-summarizer-captures-more-relevant-context-for-llm-inputs.

huggingface2024 Hugging Face, “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," February 1, 2024. Available: https://huggingface.co/papers/2401.18059.

openreview2024 OpenReview, “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," January 16, 2024. Available: https://openreview.net/forum?id=GN921JHCRw.

github2024 GitHub, “RAPTOR: The official implementation of RAPTOR for RAG Systems," Available: https://github.com/profintegra/raptor-rag, accessed October 2024.

bluebash2024 GitHub, “BlueBash/RAG-Raptor-demo," Available: https://github.com/BlueBash/RAG-Raptor-demo, accessed October 2024.

aiengineering2024 AI Engineering Academy, “RAPTOR: Recursive Abstractive Processing for Tree Organized Retrieval," Available: https://aiengineering.academy/RAG/09_RAPTOR/, accessed October 2024.

stanford2024 Stanford University, “Expanding Horizons in RAG: Exploring and Extending the Limits of RAPTOR," Available: https://web.stanford.edu/class/cs224n/final-reports/256925521.pdf, accessed October 2024.

REFERENCES

[1] S. Scott-Hayward, G. O'Callaghan **and** S. Sezer, "Sdn security: A survey," **in***2013 IEEE SDN For Future Networks and Services (SDN4FNS)* IEEE, 2013, **pages** 1–7.

[2] K. Ashton **andothers**, "That 'internet of things' thing," *RFID journal*, **jourvol** 22, **number** 7, **pages** 97–114, 2009.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐATN

A.1 Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là "Text Only" để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.2 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

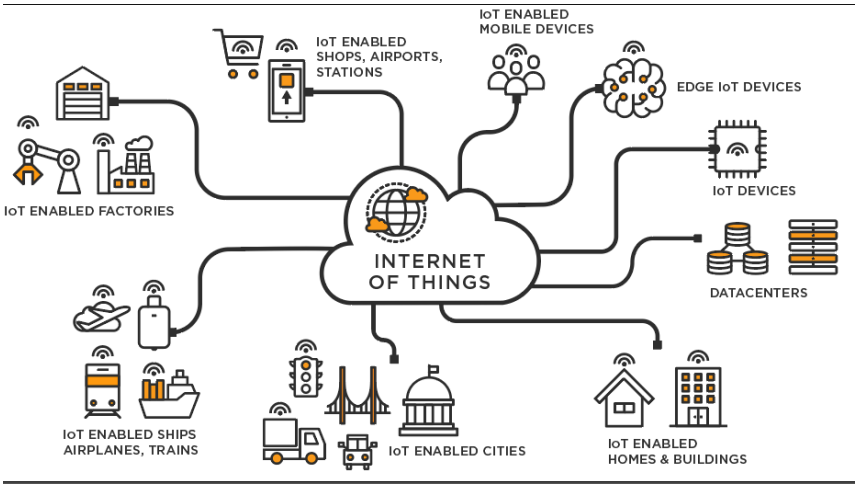
Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
  - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
  - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
  - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
  - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
  - Chương trình ICT Global: Ngành Information Technology
  - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.3 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau
  `\begin{itemize}`
  `\item` Nội dung thứ nhất được viết ở đây.
  `\item` Nội dung thứ hai được viết ở đây.
  `\item` ...
  `\end{itemize}`
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau
  `\begin{enumerate}`
  `\item` Nội dung thứ nhất được viết ở đây.
  `\item` Nội dung thứ hai được viết ở đây.
  `\item` ...



Hình A.1: Internet vạn vật

```
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại `https://www.overleaf.com/learn/latex/Lists`

A.4 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: `https://www.tablesgenerator.com/`). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link `https://www.overleaf.com/learn/latex/Tables`.

A.5 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại `https://www.overleaf.com/learn/latex/Inserting_Images`.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.6 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [1]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [2]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.7 Cách viết phương trình và công thức toán học

Các gói amsmath, amssymb, amsfonts hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file main.tex. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \tag{A.1}$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

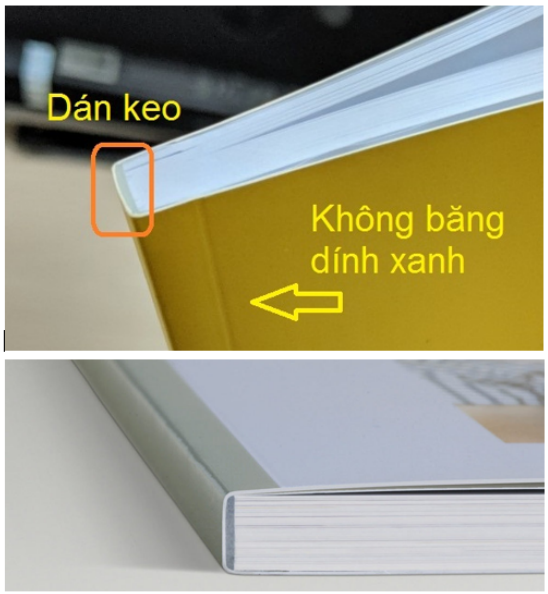
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.8 Qui cách đóng quyển

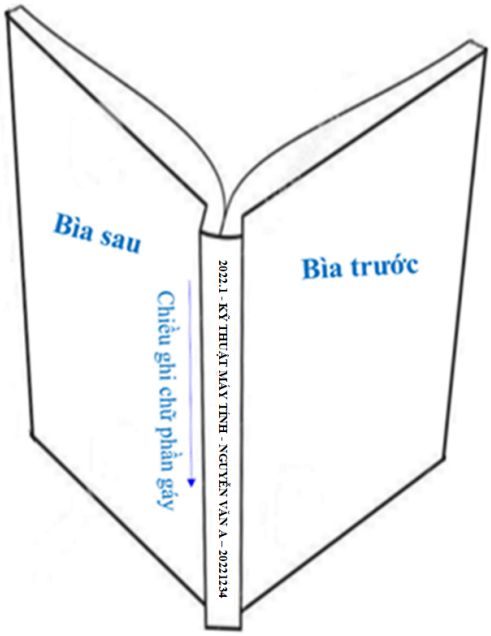
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case "Thống kê tình hình mượn sách"

...

B.2 Đặc tả use case "Đăng ký làm thẻ mượn"

...